CS0054
(INTELLIGENT SYSTEMS)

EXERCISE

4

NATURAL LANGUAGE PROCESSING (NLP)

Group Name:	Michael Dominic L. Liggayu
Members:	<table><tr><th>Name</th><th>Role</th></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr><tr><td></td><td></td></tr></table>
Section:	AN42
Professor:	Ms. Crisola G. Tan

I. PROGRAM OUTCOME/S (PO) ADDRESSED BY THE LABORATORY EXERCISE

- Design and evaluate solutions for computing problems. [PO: E]

II. COURSE LEARNING OUTCOME/S (CLO) ADDRESSED BY THE LABORATORY EXERCISE

- To design Intelligent system based on methods learned. [CLO: 3]

III. INTENDED LEARNING OUTCOME/S OF THE LABORATORY EXERCISE

At the end of this exercise, students must be able to:

- Know and understand natural language processing & language technology.
- Familiarize with the approaches to NLP.
- Learn sample applications of NLP.

IV. BACKGROUND INFORMATION

What is NLP?

Working in Data Science and having a background in Technical Writing, I was drawn to the field of Natural Language Processing (NLP). Machines understanding language fascinates me, and I often ponder which algorithms Aristotle would have used to build a rhetorical analysis machine if he had the chance. If you're new to Data Science, getting into NLP can seem complicated, especially since there have been so many recent advancements in the field. It is hard to know where to start.

What can Machines Understand?

While a computer can be actually quite good at finding patterns and summarizing documents, it must transform words into numbers before making sense of them. This transformation is needed because machines "learn" thanks to mathematics, and math doesn't work very well on words. Before transforming the words into numbers, they are often cleaned of things like special characters and punctuation, and modified into forms that make them more uniform and interpretable.

What are Word Clouds?

Word clouds (also known as text clouds or tag clouds) work in a simple way: the more a specific word appears in a source of textual data (such as a speech, blog post, or database), the bigger and bolder it appears in the word cloud.

A word cloud is a collection, or cluster, of words depicted in different sizes. The bigger and bolder the word appears, the more often it's mentioned within a given text and the more important it is.

Also known as tag clouds or text clouds, these are ideal ways to pull out the most pertinent parts of textual data, from blog posts to databases. They can also help business users compare two different pieces of text to find the wording similarities between the two.

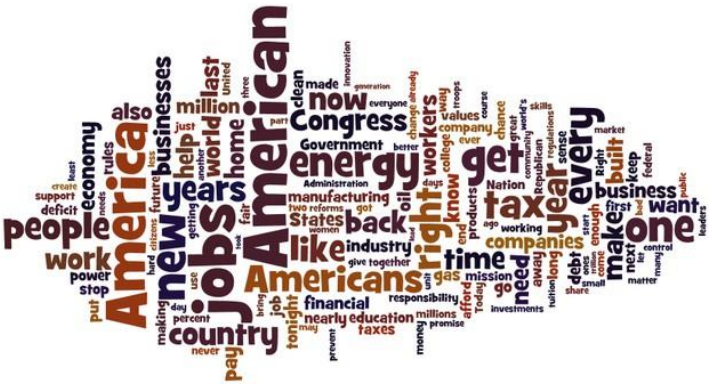
For instance, you might ask current customers what they like or don't like about your new product line. Or, you could ask them to give suggestions on how your organization could improve. They could also have the chance to elaborate on any pain points they're experiencing.

There are industry tools that allow you to code such open-ended data so users can understand it quantitatively. Yet, these don't come cheap. Word clouds offer a cost-effective, yet powerful, alternative.

With these, you can still quantify your text-based insights into measurable analytics. The only difference? You won't create a chart or graph as you would with a set of numbers.

Instead, you'll create a word cloud generator to transform the most critical information into a word cloud.

Here's an example from USA Today using U.S. President Barack Obama's State of the Union Speech 2012:



As you can see, words like "American," "jobs," "energy" and "every" stand out since they were used more frequently in the original text.

Now, compare that to the 2014 State of the Union address:



V. GRADING SYSTEM / RUBRIC (please see separate sheet)

VI. LABORATORY ACTIVITY (50 POINTS)

Data Set:

- **Spam Email Data Set found here:**
  https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb

Steps:

1. Importing Dependencies and Data

Start by importing the dependencies and the data. The data is stored as a comma separated values (csv) file, so use pandas' read_csv() function to open it into a DataFrame.

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

#create dataframe from csv
df = pd.read_csv('emails.csv')

df.head()
```

	text	spam
0	Subject: naturally irresistible your corporate...	1
1	Subject: the stock trading gunslinger fanny i...	1
2	Subject: unbelievable new homes made easy im ...	1
3	Subject: 4 color printing special request add...	1
4	Subject: do not have money , get software cds ...	1

df.head()

2. Exploratory Analysis

Before anything, it is best to do a quick analysis of the data to eliminate duplicate rows and establish some baseline counts. Use pandas drop_duplicates to drop the duplicate rows.

```
print("spam count: " +str(len(df.loc[df.spam==1])))
print("not spam count: " +str(len(df.loc[df.spam==0])))
print(df.shape)
df['spam'] = df['spam'].astype(int)

df = df.drop_duplicates()
df = df.reset_index(inplace = False)[['text','spam']]


print(df.shape)
```

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```

Counts and shape before/after duplicate removal

3. Cleaning the Texts

Notice in the word cloud image, all the text is lower case. There are no punctuation marks or special characters. That's because the text has been cleaned to make it easier to analyze. Using regular expressions, it is easy to clean the text using a loop:

```
clean_desc = []
for w in range(len(df.text)):
    desc = df['text'][w].lower()

    #remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    #remove tags
    desc=re.sub("&lt;/?.*?&gt;"," &lt;&gt; ",desc)

    #remove digits and special chars
    desc=re.sub("(\\d|\\W)+"," ",desc)

    clean_desc.append(desc)


#assign the cleaned descriptions to the data frame
df['text'] = clean_desc

df.head(3)
```

	text	spam
0	subject naturally irresistible your corporate ...	1
1	subject the stock trading gunslinger fanny is ...	1
2	subject unbelievable new homes made easy im wa...	1

4. Removing Stop Words

Stop words are the most common words like “the” and “of.” Removing them from the email text allows the more relevant frequent words to stand out. Removing stop words is a common technique! Some Python libraries like NLTK come pre-loaded with a list of stop words, but it is easy to create one from scratch.

```
stop_words = ['is','you','your','and', 'the', 'to', 'from', 'or',
'I', 'for', 'do', 'get', 'not', 'here', 'in', 'im', 'have', 'on',
're', 'new', 'subject']
```

5. Construct the Word Cloud

Conveniently, there is a Python library for creating word clouds. It can be installed using pip.

```
pip install wordcloud
```

When constructing the word cloud, it is possible to set several parameters like height and width, stop words, and max words. It is even possible to shape it instead of displaying the default rectangle.

```
wordcloud = WordCloud(width = 800, height = 800, background_color =
'black', stopwords = stop_words, max_words = 1000
                      , min_font_size =
20).generate(str(df1['text']))

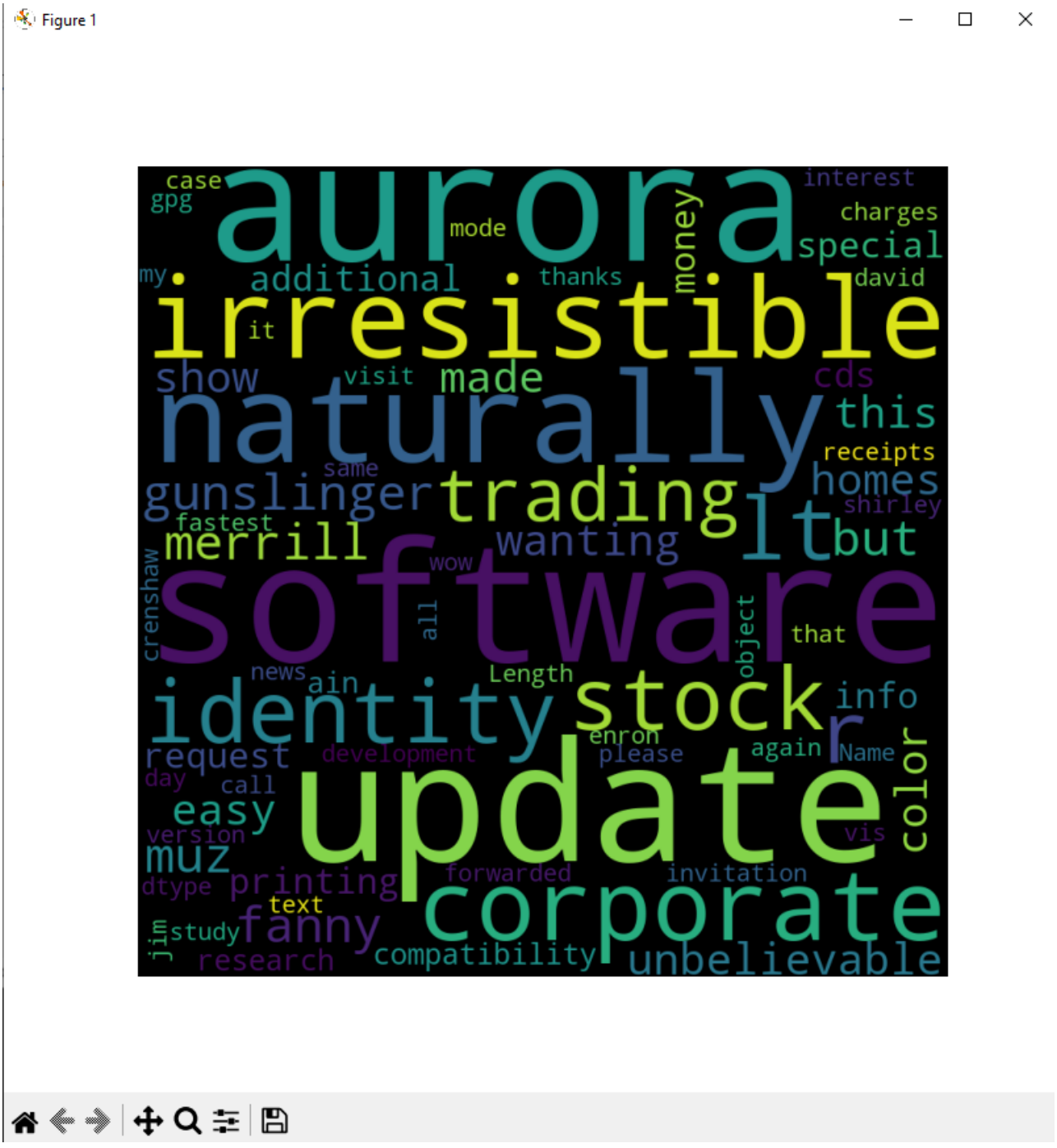

#plot the word cloud
fig = plt.figure(figsize = (8,8), facecolor = None)
plt.imshow(wordcloud)
plt.axis('off')
plt.show()
```

The word cloud can be saved and displayed using matplotlib and .show(). This is the result of all records, regardless of it being spam.



OUTPUT:

```
spam count: 1368
not spam count: 4360
(5728, 2)
(5695, 2)
```


Figure 1
aurora
irresistible
naturally
software
update
corporate
identity
stock
trading
unbelievable
compatibility
research
fanny
study
printing
muz
easy
request
merrill
gunslinger
fastest
wanting
but
lt
homes
this
receipts
cds
made
show
visit
additional
thanks
money
special
charges
interest
mode
gpg
case
my
it
david
shirley
same
wow
all
crenshaw
that
object
Length
news
ain
info
r
color
enron
please
again
Name
development
day
call
version
vis
dtype
forwarded
invitation
text
jim

CODE:

```
# 1. Importing Dependencies and Data

import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# Create dataframe from CSV
df = pd.read_csv('emails.csv')

df.head()


# 2. Exploratory Analysis

print("spam count: " + str(len(df.loc[df.spam == 1])))
print("not spam count: " + str(len(df.loc[df.spam == 0])))

print(df.shape)

df['spam'] = df['spam'].astype(int)

# Remove duplicate rows
df = df.drop_duplicates()

df = df.reset_index(inplace=False)[['text', 'spam']]

print(df.shape)


# 3. Cleaning the Texts

clean_desc = []

for w in range(len(df.text)):
    desc = df['text'][w].lower()

    # Remove punctuation
    desc = re.sub('[^a-zA-Z]', ' ', desc)

    # Remove tags
    desc = re.sub("&lt;/?.*?&gt;", " ", desc)

    # Remove digits and special characters
    desc = re.sub(r"(\d|\W)+", " ", desc)

    clean_desc.append(desc)
```

```
# Assign the cleaned descriptions to the dataframe
df['text'] = clean_desc

df.head(3)


# 4. Removing Stop Words

stop_words = [
    'is', 'you', 'your', 'and', 'the', 'to', 'from',
    'or', 'I', 'for', 'do', 'get', 'not', 'here',
    'in', 'im', 'have', 'on', 're', 'new', 'subject'
]

# Remove stop words from the text
df['text'] = df['text'].apply(
    lambda x: ' '.join(
        [word for word in x.split() if word not in stop_words]
    )
)


# 5. Construct the Word Cloud

wordcloud = WordCloud(
    width=800,
    height=800,
    background_color='black',
    stopwords=stop_words,
    max_words=1000,
    min_font_size=20
).generate(str(df['text']))

# Plot the word cloud
fig = plt.figure(figsize=(8, 8), facecolor=None)

plt.imshow(wordcloud)
plt.axis('off')
plt.show()

# Save the generated word cloud
wordcloud.to_file("spam_wordcloud.png")

print("Word Cloud successfully generated!")
```

CODE SCREENSHOT:

TA9_Liggayu_Part1.py > ...

```
# 1. Importing Dependencies and Data

import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud

# Create dataframe from CSV
df = pd.read_csv('emails.csv')

df.head()


# 2. Exploratory Analysis

print("spam count: " + str(len(df.loc[df.spam == 1])))
print("not spam count: " + str(len(df.loc[df.spam == 0])))

print(df.shape)

df['spam'] = df['spam'].astype(int)

# Remove duplicate rows
df = df.drop_duplicates()

df = df.reset_index(inplace=False)[['text', 'spam']]

print(df.shape)


# 3. Cleaning the Texts

clean_desc = []

for w in range(len(df.text)):
    desc = df['text'][w].lower()
```

```
38
39      # Remove punctuation
40      desc = re.sub('[^a-zA-Z]', ' ', desc)
41
42      # Remove tags
43      desc = re.sub("&lt;/?.*?&gt;", " ", desc)
44
45      # Remove digits and special characters
46      desc = re.sub(r"(\d|\W)+", " ", desc)
47
48      clean_desc.append(desc)
49
50  # Assign the cleaned descriptions to the dataframe
51  df['text'] = clean_desc
52
53  df.head(3)
54
55
56  # 4. Removing Stop Words
57
58  stop_words = [
59      'is', 'you', 'your', 'and', 'the', 'to', 'from',
60      'or', 'I', 'for', 'do', 'get', 'not', 'here',
61      'in', 'im', 'have', 'on', 're', 'new', 'subject'
62  ]
63
64  # Remove stop words from the text
65  df['text'] = df['text'].apply(
66      lambda x: ' '.join(
67          [word for word in x.split() if word not in stop_words]
68      )
69  )
70
```

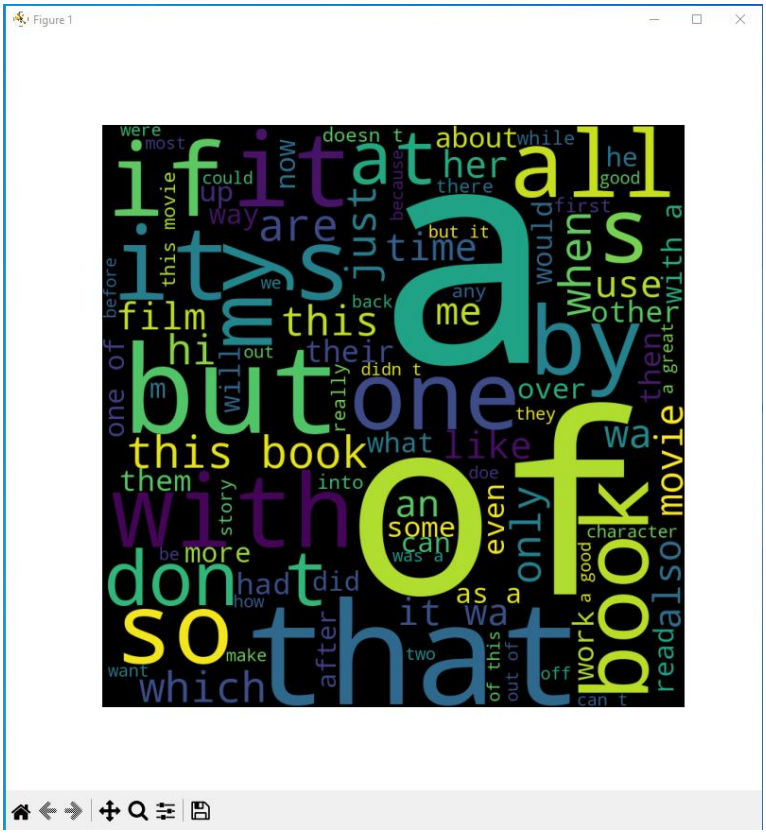
```
71
72  # 5. Construct the Word Cloud
73
74  wordcloud = WordCloud(
75      width=800,
76      height=800,
77      background_color='black',
78      stopwords=stop_words,
79      max_words=1000,
80      min_font_size=20
81  ).generate(str(df['text']))
82
83  # Plot the word cloud
84  fig = plt.figure(figsize=(8, 8), facecolor=None)
85
86  plt.imshow(wordcloud)
87  plt.axis('off')
88  plt.show()
89
90  # Save the generated word cloud
91  wordcloud.to_file("spam_wordcloud.png")
92
93  print("Word Cloud successfully generated!")
94
```

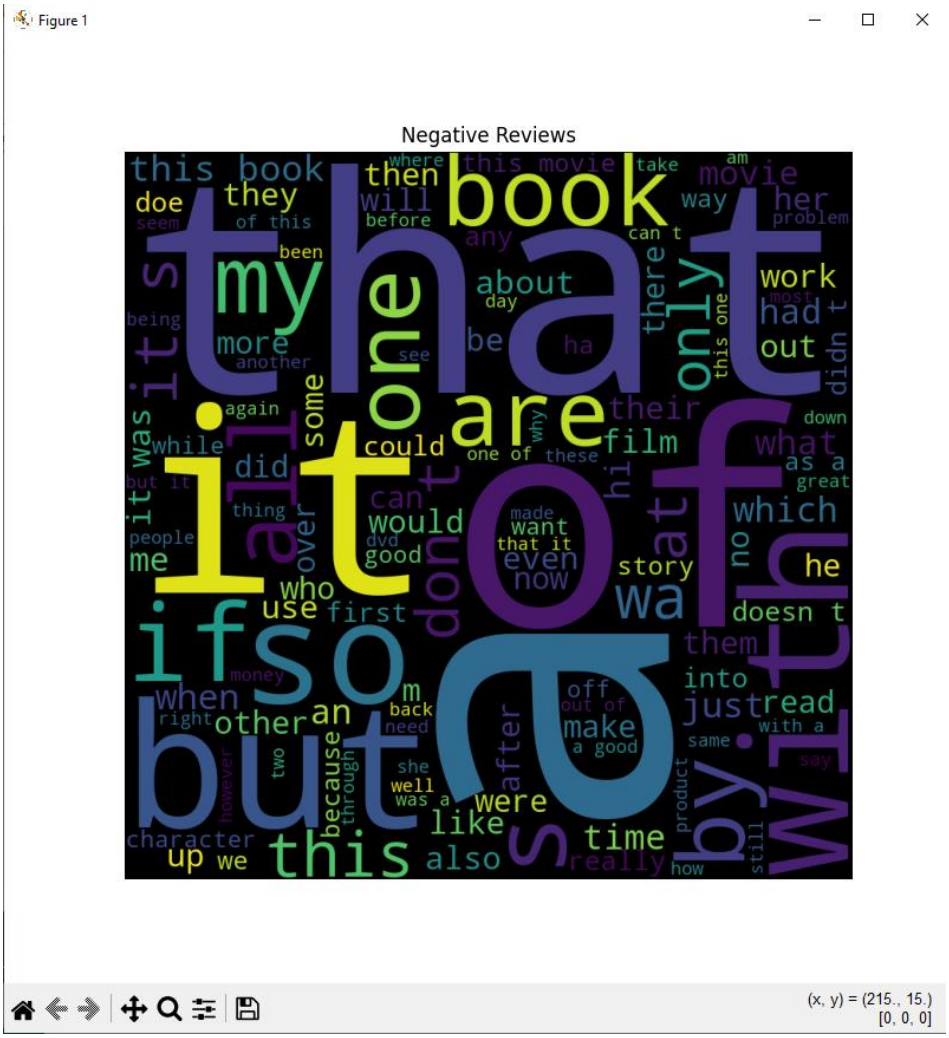
VII. CHALLENGE

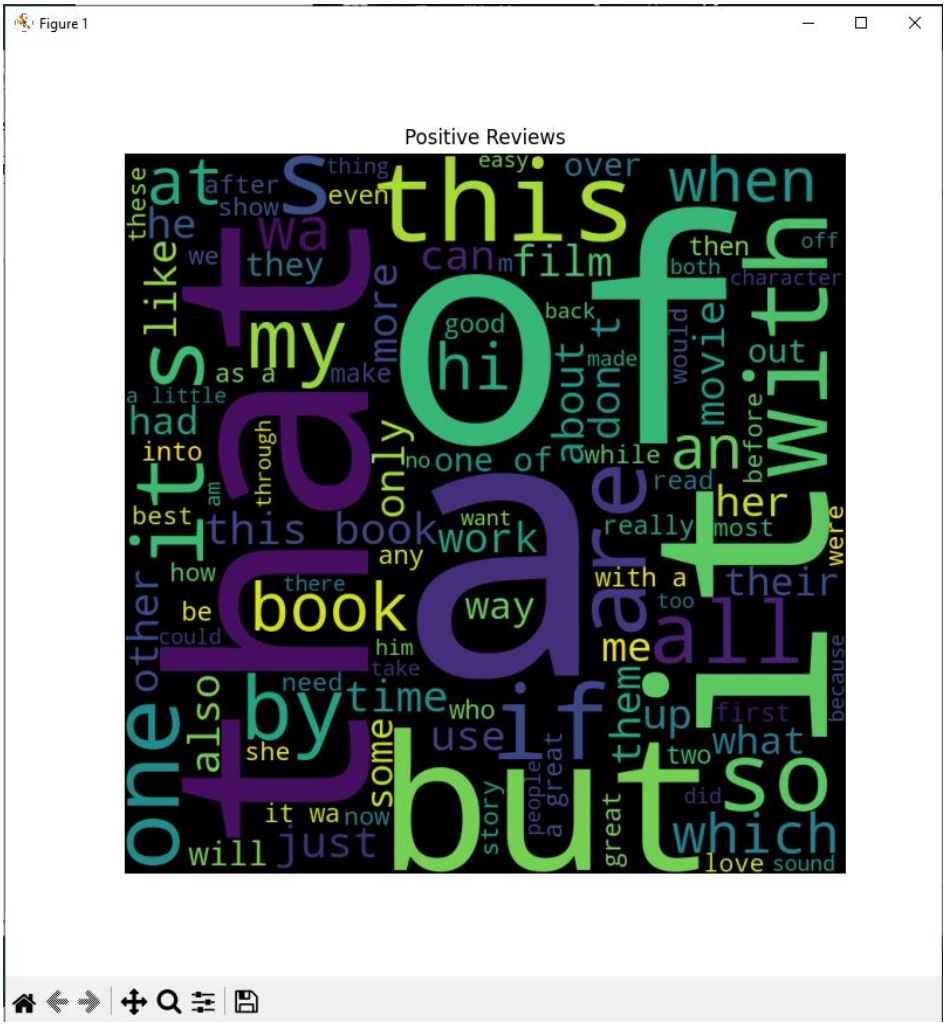
- Repeat the procedure you did in Part VI., this time using any of the following open data sets found in this URL: https://analyticsindiamag.com/10-nlp-open-source-datasets-to-start-your-first-nlp-project/
- Clearly specify the intention of the output that you will generate.
- Compare and contrast sample outputs that you will generate.
- Provide a conclusion on whether you have achieved your goal or not.
- Provide a recommendation on things you did wrong and suggestions to improve and achieve the expected output.
- You may use existing Word Cloud generator like Wordle, Tagxedo, Tag Crowd, WordItOut, Word Cloud Generator for Chrome, Word Cloud Python tools or Google Word Cloud tools and check your output against the output of the generators. Analyze and explain results.

OUTPUT:

```
ear\first sem\CS Elective - Intelligent Syste
Positive reviews: 4000
Negative reviews: 4000
Dataset shape: (8000, 2)
Shape after removing duplicates: (7853, 2)
```




Figure 1
Negative Reviews
(x, y) = (215., 15.)
[0, 0, 0]



CODE:

```
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from pathlib import Path

# Import the dataset folder
dataset_path = Path("sorted_data_acl")

# Read positive and negative review files
reviews = []

for file_path in dataset_path.rglob("*.review"):
```

```
        sentiment = file_path.stem

        if sentiment not in ["positive", "negative"]:
            continue

        with open(file_path, "r", encoding="latin-1") as file:
            content = file.read()

        # Extract review texts
        review_texts = re.findall(
            r"<review_text>(.*?)</review_text>",
            content,
            flags=re.DOTALL | re.IGNORECASE
        )

        for review in review_texts:
            reviews.append({
                "text": review,
                "sentiment": sentiment
            })

# Create dataframe
df = pd.DataFrame(reviews)

# Exploratory analysis
print("Positive reviews:", len(df[df.sentiment == "positive"]))
print("Negative reviews:", len(df[df.sentiment == "negative"]))
print("Dataset shape:", df.shape)

# Remove duplicate records
df = df.drop_duplicates()
df = df.reset_index(drop=True)

print("Shape after removing duplicates:", df.shape)

# Clean the texts
clean_desc = []

for w in range(len(df.text)):

    desc = df["text"][w].lower()

    # Remove punctuation
    desc = re.sub("[^a-zA-Z]", " ", desc)

    # Remove tags
    desc = re.sub("&lt;/?.*?&gt;", " ", desc)

    # Remove digits and special characters
    desc = re.sub(r"(\d|\W)+", " ", desc)
```

```
    clean_desc.append(desc)

# Assign cleaned descriptions to dataframe
df["text"] = clean_desc

# Remove stop words
stop_words = [
    "is", "you", "your", "and", "the", "to", "from", "or",
    "i", "for", "do", "get", "not", "here", "in", "im",
    "have", "on", "re", "new", "subject"
]

# Generate Word Cloud for all reviews
wordcloud = WordCloud(
    width=800,
    height=800,
    background_color="black",
    stopwords=stop_words,
    max_words=1000,
    min_font_size=20
).generate(" ".join(df["text"]))

# Display the Word Cloud
fig = plt.figure(figsize=(8, 8), facecolor=None)
plt.imshow(wordcloud)
plt.axis("off")
plt.savefig("challenge_wordcloud.png", bbox_inches="tight")
plt.show()

# Generate separate positive and negative Word Clouds
for sentiment in ["positive", "negative"]:

    text_data = " ".join(
        df.loc[df["sentiment"] == sentiment, "text"]
    )

    if text_data.strip():

        wordcloud = WordCloud(
            width=800,
            height=800,
            background_color="black",
            stopwords=stop_words,
            max_words=1000,
            min_font_size=20
        ).generate(text_data)

        plt.figure(figsize=(8, 8))
        plt.imshow(wordcloud)
        plt.axis("off")
        plt.title(sentiment.capitalize() + " Reviews")
```

```
plt.savefig(
    sentiment + "_wordcloud.png",
    bbox_inches="tight"
)
plt.show()
```

CODE SCREENSHOT:

TA9_Liggayu_Part2.py > ...

```python
import pandas as pd
import sqlite3
import regex as re
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from pathlib import Path

# Import the dataset folder
dataset_path = Path("sorted_data_acl")

# Read positive and negative review files
reviews = []

for file_path in dataset_path.rglob("*.review"):

    sentiment = file_path.stem

    if sentiment not in ["positive", "negative"]:
        continue

    with open(file_path, "r", encoding="latin-1") as file:
        content = file.read()

    # Extract review texts
    review_texts = re.findall(
        r"<review_text>(.*?)</review_text>",
        content,
        flags=re.DOTALL | re.IGNORECASE
    )

    for review in review_texts:
        reviews.append({
            "text": review,
            "sentiment": sentiment
        })

# Create dataframe
df = pd.DataFrame(reviews)

# Exploratory analysis
print("Positive reviews:", len(df[df.sentiment == "positive"]))
print("Negative reviews:", len(df[df.sentiment == "negative"]))
print("Dataset shape:", df.shape)

# Remove duplicate records
df = df.drop_duplicates()
df = df.reset_index(drop=True)
```

```
49
50  print("Shape after removing duplicates:", df.shape)
51
52  # Clean the texts
53  clean_desc = []
54
55  for w in range(len(df.text)):
56
57      desc = df["text"][w].lower()
58
59      # Remove punctuation
60      desc = re.sub("[^a-zA-Z]", " ", desc)
61
62      # Remove tags
63      desc = re.sub("&lt;/?.*?&gt;", " ", desc)
64
65      # Remove digits and special characters
66      desc = re.sub(r"(\d|\W)+", " ", desc)
67
68      clean_desc.append(desc)
69
70  # Assign cleaned descriptions to dataframe
71  df["text"] = clean_desc
72
73  # Remove stop words
74  stop_words = [
75      "is", "you", "your", "and", "the", "to", "from", "or",
76      "i", "for", "do", "get", "not", "here", "in", "im",
77      "have", "on", "re", "new", "subject"
78  ]
79
80  # Generate Word Cloud for all reviews
81  wordcloud = WordCloud(
82      width=800,
83      height=800,
84      background_color="black",
85      stopwords=stop_words,
86      max_words=1000,
87      min_font_size=20
```

```
).generate(" ".join(df["text"]))

# Display the Word Cloud
fig = plt.figure(figsize=(8, 8), facecolor=None)
plt.imshow(wordcloud)
plt.axis("off")
plt.savefig("challenge_wordcloud.png", bbox_inches="tight")
plt.show()

# Generate separate positive and negative Word Clouds
for sentiment in ["positive", "negative"]:

    text_data = " ".join(
        df.loc[df["sentiment"] == sentiment, "text"]
    )

    if text_data.strip():

        wordcloud = WordCloud(
            width=800,
            height=800,
            background_color="black",
            stopwords=stop_words,
            max_words=1000,
            min_font_size=20
        ).generate(text_data)

        plt.figure(figsize=(8, 8))
        plt.imshow(wordcloud)
        plt.axis("off")
        plt.title(sentiment.capitalize() + " Reviews")
        plt.savefig(
            sentiment + "_wordcloud.png",
            bbox_inches="tight"
        )
        plt.show()

```

A. Intention:

The intention of this program is to apply Natural Language Processing techniques particulary to the Multi-Domain Sentiment Dataset. The program identifies frequently used words in Amazon product reviews and visualize using Word Clouds. It also compares positive and negative reviews to identify differences in commonly used words.

B. Compare and Contrast

The Word Cloud displays frequently used words from both positive and negative reviews. The positive word cloud focuses on words more on favorable products while the negative Word Cloud focuses on words more on unfavorable reviews.

C. Conclusion

The program applies the NLP techniques to another text dataset and generate using Word Clouds. Additionally, the output provides a visual representation of frequently used words allowing a comparison between the positive and negative product reviews.

D. Recommendation

I recommend adding more stop words to remove unnecessary words and make the Word Clouds clearer. I also suggest analyzing each product category seperately to get better results and understand customer reviews more easily.

VIII. REFERENCES

- Vahid Mirjalili(2017). Python Machine Learning: Machine Learning and Deep Learning with Python, scikit-learn, and TensorFlow. India: Packt Publishing
- Numerical Algorithms: Methods for Computer Vision, Machine Learning, and Graphics

Online Sources:

- https://boostlabs.com/blog/what-are-word-clouds-value-simple-visualizations/
- https://github.com/bendgame/nlpBeginnerProjects/blob/master/NLP_beginner_projects.ipynb
- https://towardsdatascience.com/3-super-simple-projects-to-learn-natural-language-processing-using-python-8ef74c757cd9